

# The SPO tree: the core algorithm, block by block

Growth and pruning as pseudo-code, as `spo_tree.py` runs them

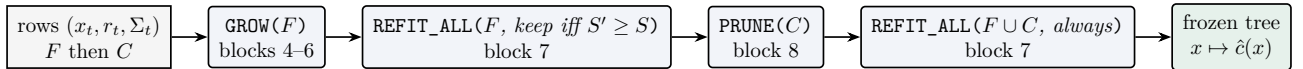
25 September 2026

## 0. Notation and state

| symbol                                                         | meaning                                                                                                                                                                                                  |
|----------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $t = 1..T$                                                     | decision dates, in time order ; $F$ = the first 75% (fitting rows), $C$ = the last 25% (checking rows)                                                                                                   |
| $x_t, r_t, \Sigma_t$                                           | features on date $t$ ; $h$ -day simple returns of the $n$ assets (cash = 0) ; $h$ -day covariance from the past only                                                                                     |
| $w^*(c; u, \Sigma)$                                            | the optimizer: $\arg \max_{w \in \mathcal{W}} c^\top w - f^\top  w - u  - \lambda w^\top \Sigma w$ , $\mathcal{W} = \{0 \leq w_i \leq \bar{w}, \mathbf{1}^\top w = 1\}$ , solved exactly because concave |
| $\text{utility}(w; r, u, \Sigma)$                              | $r^\top w - f^\top  w - u  - \lambda w^\top \Sigma w$                                                                                                                                                    |
| $U[t], B[t]$                                                   | training state, one row per date: holdings before decision $t$ ; hindsight utility from those holdings. Set by <b>MEASURE</b> , frozen between two calls                                                 |
| leaf $\ell = (\text{node}, \text{rows}, \omega, \text{depth})$ | node: score $\hat{c}_\ell \in [-b, b]^n$ , mass $n_\ell$ , regret sum $L_\ell$ ; rows: the dates reaching $\ell$ ; $\omega$ : their weights                                                              |
| split node                                                     | feature $j$ , threshold $\theta$ , two children ; a soft node also holds thresholds $\theta_1 < \dots < \theta_K$ and weights $\pi_k$                                                                    |
| $S$                                                            | in-sample Sharpe ratio of the current tree's replayed path                                                                                                                                               |

**Parameters of v1:**  $\lambda = 1$ ,  $f = (3 \text{ bp}, 0)$ ,  $\bar{w} = 1$ ,  $b = 10\%$ ,  $m$  = minimum number of rows in a leaf = 20,  $\varphi$  = minimum fraction of rows from the parent in a leaf = 0.10,  $D$  = maximum depth = 3, shortlist splits (thresholds) 3,  $\varepsilon_R$  = minimum regret loss improvement =  $10^{-6}$ ,  $\varepsilon_S$  = minimum sharpe improvement = 0,  $k_p$  = pruning hurdle = 2,  $D_p$  = pruning depth = 1, deciles + refinement with  $k = 1$  (refinement hurdle: the refined threshold replaces the best decile only if its regret is lower by more than  $k$  standard errors of the difference),  $\kappa$  = soft-split temperature = 0.

## 1 MAIN




---

### Algorithm 1: MAIN: SPOPortfolioTree.fit

---

**Input:** rows  $t = 1..T$ ;  $T_f = 0.75T$ ;  $F = \{1, \dots, T_f\}$ ,  $C = \{T_f + 1, \dots, T\}$

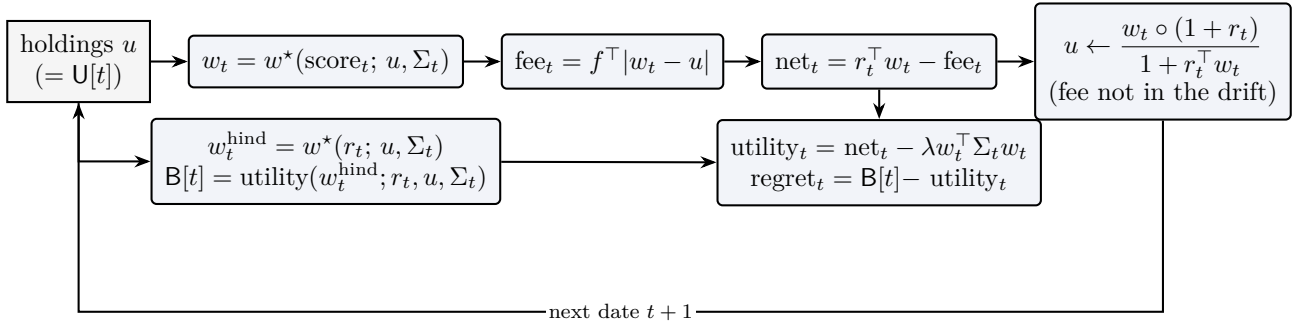
- 1 root, leaves, path  $\leftarrow$  **GROW**( $F$ );
- 2 **REFIT\_ALL**(leaves, path, rows =  $F$ , always = false);
- 3 **PRUNE**(root);
- 4 route all rows  $F \cup C$  through the pruned tree; path  $\leftarrow$  **REPLAY**(**SCORES**(leaves),  $F \cup C$ ,  $u_1 = \mathbf{1}/n$ ); **MEASURE**(path);
- 5 **REFIT\_ALL**(leaves, path, rows =  $F \cup C$ , always = true);

**Output:** the tree; in use:  $w_t = w^*(\hat{c}(x_t); u_t, \Sigma_t)$  with the live holdings  $u_t$

---

## 2 REPLAY: the path, the fees, the row regrets

Fees are computed here and nowhere else. One date:




---

### Algorithm 2: REPLAY(scores, rows, $u_1$ ) (replay\_path) and MEASURE

---

```

1   $u \leftarrow u_1$ ;
2  for  $t$  in rows, in time order do
3     $U[t] \leftarrow u$ ;  $w_t \leftarrow w^*(\text{scores}_t; u, \Sigma_t)$ ;
4     $\text{fee}_t \leftarrow f^T |w_t - u|$ ;  $\text{net}_t \leftarrow r_t^T w_t - \text{fee}_t$ ;
5     $u \leftarrow w_t \circ (1 + r_t) / (1 + r_t^T w_t)$ ;
6   $\text{utility}_t \leftarrow r_t^T w_t - \text{fee}_t - \lambda w_t^T \Sigma_t w_t$ ;  $B[t] \leftarrow \text{utility}(w^*(r_t; U[t], \Sigma_t); r_t, U[t], \Sigma_t)$ ;
    $\text{regret}_t \leftarrow B[t] - \text{utility}_t$ ;
7   $S \leftarrow \text{mean}(\text{net}) / \text{std}(\text{net})$ ; // 0 if net  $\equiv$  0
8  return path = (U, w, net, regret, S, final u);
9  function MEASURE(path):
10   U  $\leftarrow$  path.U; B  $\leftarrow$  path.utility + path.regret; S  $\leftarrow$  path.S; // holdings frozen from
   here

```

---

## 3 The row regret and the score search of a leaf

---

### Algorithm 3: REG( $c, t$ ) (\_regret\_rows) and FIT\_LEAF(rows, $\omega$ , fallback) (\_fit\_leaf)

---

```

1  function REG( $c, t$ ):
2     $w \leftarrow w^*(c; U[t], \Sigma_t)$ ; return max (B[t] - utility( $w; r_t, U[t], \Sigma_t$ ), 0); // abort if
    $< -10^{-6}$ 
3   $L(c) := \sum_{t \in \text{rows}} \omega_t \text{REG}(c, t)$ ; // leaf regret of a score vector
4  function FIT_LEAF(rows,  $\omega$ , fallback):
5     $c \leftarrow \text{clip}(\sum_t \omega_t r_t / \sum_t \omega_t, -b, b)$ ; // start: the weighted mean return
6    if fallback given and  $L(\text{fallback}) < L(c)$  then
7       $c \leftarrow \text{fallback}$ 
8    radius  $\leftarrow 2b$ ;
9    for pass = 1, 2, 3 do
10     for coord in (asset, cash) do
11       grid  $\leftarrow$  7 equally spaced values in  $[c_{\text{coord}} - \text{radius}, c_{\text{coord}} + \text{radius}] \cap [-b, b]$ ;
       // pass 1: all of  $[-b, b]$ 
12        $c_{\text{coord}} \leftarrow$  the grid value with the smallest L, if strictly smaller than the current L(c);
13       radius  $\leftarrow$  radius/6;
14   return node( $\hat{c} = c, n = \sum \omega, L = L(c)$ );

```

---

Cost:  $2 + 3 \cdot 7 \cdot n$  evaluations of  $L$ , each one vectorised optimizer call over the leaf's rows. Only score differences matter, so the cash coordinate is redundant; cash is reported as 0.

## 4 GROW

---

### Algorithm 4: GROW( $F$ ) ( $\_grow$ )

---

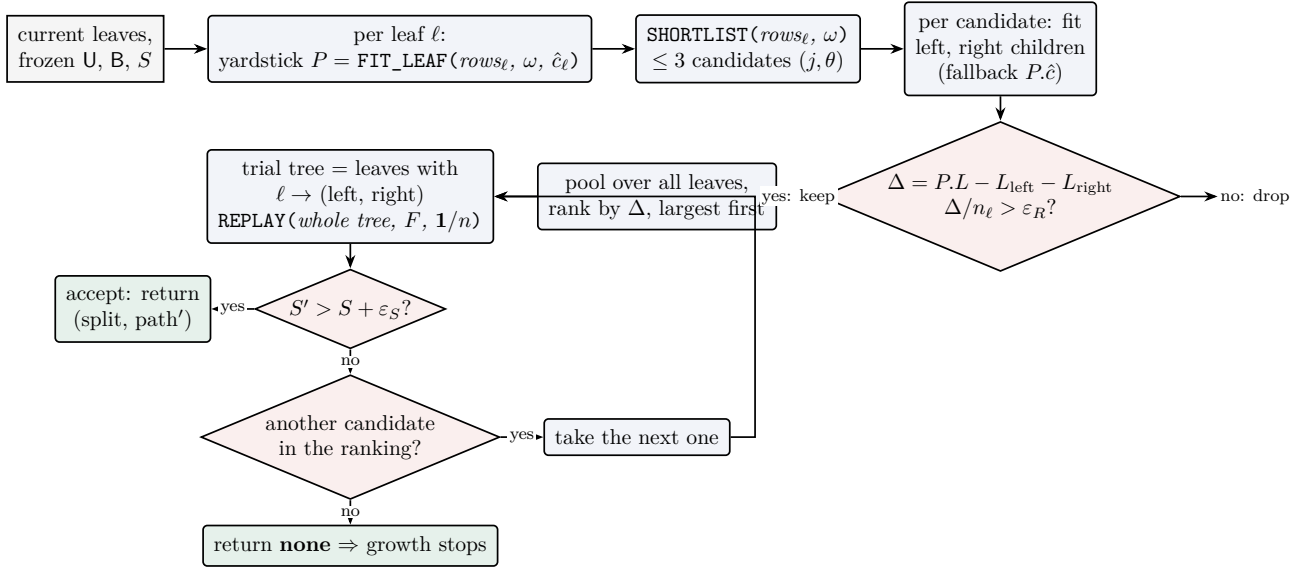
```

1  $U[t] \leftarrow 1/n$ ,  $B[t] \leftarrow \text{utility}(w^*(r_t; 1/n, \Sigma_t); r_t, 1/n, \Sigma_t)$  for all  $t \in F$  ;    // convention: no
  path exists yet
2  $\text{root} \leftarrow \text{FIT\_LEAF}(F, \omega \equiv 1, \text{none})$ ;  $\text{leaves} \leftarrow [(\text{root}, F, 1, \text{depth } 0)]$ ;
3  $\text{path} \leftarrow \text{REPLAY}(\text{SCORES}(\text{leaves}), F, 1/n)$ ;  $\text{MEASURE}(\text{path})$ ;
4 while true do
5    $\text{result} \leftarrow \text{BEST\_SPLIT}(\text{leaves}, S)$  ;                                     // block 5
6   if  $\text{result} = \text{none}$  then
7     break
8    $\text{leaves} \leftarrow \text{leaves} - \{\ell\} + \{\text{left child}, \text{right child}\}$ ; node of  $\ell$  becomes a split node  $(j, \theta)$ ;
9    $\text{path} \leftarrow \text{result.path}$ ;  $\text{MEASURE}(\text{path})$  ; // new holdings, new  $S$ ; children keep the
    rows of step 5
10 return  $\text{root}$ ,  $\text{leaves}$ ,  $\text{path}$ ;
```

---

$\text{SCORES}(\text{leaves})$ : the per-row score matrix, row  $t = \sum_{\ell} \omega_{t\ell} \hat{c}_{\ell}$  (one leaf per row with plain splits).

## 5 BEST\_SPLIT: one round



---

**Algorithm 5: BEST\_SPLIT(leaves,  $S$ ) (\_best\_split)**

---

```
1 cand  $\leftarrow []$ ;
2 foreach leaf  $\ell = (node, rows, \omega, depth)$  do
3   if  $depth \geq D$  or  $\sum \omega < 2m_*$ ,  $m_* = \max(m, \lceil \varphi \sum \omega \rceil)$  then
4     skip
5    $P \leftarrow \text{FIT\_LEAF}(rows, \omega, node.\hat{c})$  ; // same rows, current U: the yardstick; never
      written into the tree
6   foreach  $(j, \theta)$  in  $\text{SHORTLIST}(rows, \omega)$  do
7     left  $\leftarrow \{t \in rows : x_{tj} \leq \theta\}$ ; right  $\leftarrow rows \setminus left$ ;
8      $Lc \leftarrow \text{FIT\_LEAF}(left, \omega, P.\hat{c})$ ;  $Rc \leftarrow \text{FIT\_LEAF}(right, \omega, P.\hat{c})$ ;
9      $\Delta \leftarrow P.L - Lc.L - Rc.L$ ;
10    if  $\Delta / \sum \omega > \varepsilon_R$  then
11      if  $\Delta / \sum \omega > \varepsilon_R$  then
12        cand  $\leftarrow cand + (\Delta, \ell, j, \theta, Lc, Rc)$  ; // regret gate: local, U frozen
13  sort cand by  $\Delta$ , largest first ; // SPOT's criterion, over all leaves
14  foreach  $(\Delta, \ell, j, \theta, Lc, Rc)$  in cand do
15    trial  $\leftarrow$  leaves with  $\ell$  replaced by  $(Lc, \text{left rows}), (Rc, \text{right rows})$  ; // other leaves
      unchanged
16    path'  $\leftarrow \text{REPLAY}(\text{SCORES}(trial), F, 1/n)$  ; // whole tree, whole fitting period,
      fees included
17    if  $path'.S > S + \varepsilon_S$  then
18      return (split, path') ; // Sharpe gate: in-sample; one acceptance per round
19 return none;
```

---

With plain splits the memberships are 0/1: left rows get  $\omega$ , right rows get  $\omega$ . With soft splits ( $\kappa > 0$ ) a row goes left with weight  $\mu(x) = \sum_{k:\theta_k \geq x} \pi_k$  and right with  $1 - \mu(x)$ , and a row can be in both children.

## 6 SHORTLIST and SCREEN: the candidates of one leaf

---

**Algorithm 6:** SHORTLIST( $rows, \omega$ ) (`_shortlist`), v1 settings; the soft variant in the last lines

---

```

1  $m_* \leftarrow \max(m, \lceil \varphi \sum \omega \rceil)$ ; screened  $\leftarrow []$ ;
2 foreach feature  $j$  do
3    $\Theta \leftarrow$  midpoints of consecutive distinct values of  $x_{.j}$  on rows with mass  $\geq m_*$  on each side ;
   // plain candidates  $\theta_{(1)} < \dots < \theta_{(K)}$ 
4   if deciles then
5      $\Theta \leftarrow \Theta[\text{round}(q(K-1))]$  for  $q = 0.1, \dots, 0.9$  ; // quantiles by rank among the
     admissible thresholds
6    $Q, \text{per\_row} \leftarrow \text{SCREEN}(rows, \omega, x_{.j}, \Theta)$ ;
7   if refinement then
8      $\theta_o \leftarrow \arg \min_{\Theta} Q$ ;  $W \leftarrow$  plain thresholds strictly between the two elements of  $\Theta$ 
     neighbouring  $\theta_o$ ;
9      $Q_W, \text{per\_row}_W \leftarrow \text{SCREEN}(rows, \omega, x_{.j}, W)$ ;  $\theta_{\bullet} \leftarrow \arg \min_W Q_W$ ;
10     $se \leftarrow \sqrt{|rows|} \cdot \text{std}_t(\text{per\_row}_W[\theta_{\bullet}] - \text{per\_row}[\theta_o])$ ;
11    if  $Q(\theta_o) - Q_W(\theta_{\bullet}) > k \cdot se$  then
12      replace  $\theta_o$  by  $\theta_{\bullet}$  in  $\Theta$  (and its  $Q$ )
13  if  $\kappa = 0$  (plain) then
14    screened  $\leftarrow$  screened  $+\{(Q(\theta), j, \theta) : \theta \in \Theta\}$ 
15  else
16     $z_k \leftarrow (Q(\theta_k) - \min_l Q(\theta_l))/se_k$ ;  $\pi_k \propto e^{-z_k/\kappa}$  (or  $e^{-z_k^2/2\kappa^2}$ ); screened  $\leftarrow$  screened
     $+\{(\min Q, j, \Theta, \pi)\}$  ; // one soft split per feature
17 return the 3 entries of screened with the smallest  $Q$ ;
18 function SCREEN( $rows, \omega, values, \Theta$ ):
19   foreach  $\theta \in \Theta$  do
20     foreach side in ( $values \leq \theta, values > \theta$ ) do
21        $c_{\text{side}} \leftarrow$  clip(weighted mean of  $r_t$  on the side,  $\pm b$ );  $\text{regret}_t \leftarrow \omega_t \text{REG}(c_{\text{side}}, t)$  for  $t$  on
       the side;
22      $Q(\theta) \leftarrow \sum_t \text{regret}_t$ ;  $\text{per\_row}[\theta] \leftarrow (\text{regret}_t)_t$ ;
23   return  $Q, \text{per\_row}$ ;

```

---

## 7 REFIT\_ALL

---

**Algorithm 7:** REFIT\_ALL( $leaves, path, rows, always$ ) (`_refit_leaves`)

---

```

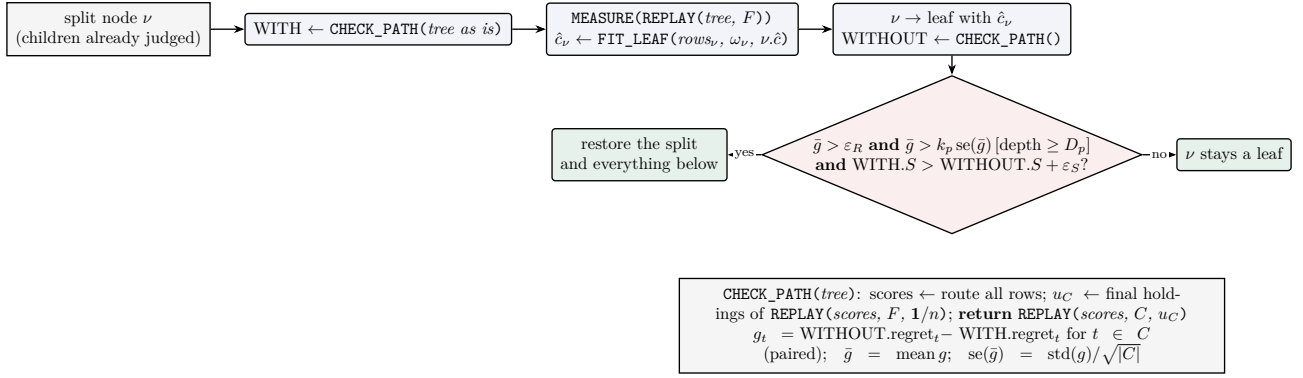
1 foreach leaf  $\ell$  do
2    $\hat{c}'_{\ell} \leftarrow \text{FIT\_LEAF}(rows_{\ell}, \omega_{\ell}, \hat{c}_{\ell})$  ; // from the current  $\mathcal{U}$  (the path of the final
   tree)
3  $path' \leftarrow \text{REPLAY}(scores \text{ with } \hat{c}', rows, 1/n)$ ;
4 if always or  $path'.S \geq path.S$  then
5   write every  $\hat{c}'_{\ell}$  into its leaf ; // all or nothing

```

---

Step 4 (after growth):  $rows = F$ , kept only if  $S$  does not fall. Step 6 (after pruning):  $rows = F \cup C$  routed through the pruned tree, MEASURE first, always kept: the scores it replaces never saw  $C$ .

## 8 PRUNE: bottom-up on the checking rows




---

### Algorithm 8: PRUNE(root) (\_prune)

---

```

1 function VISIT(ν, rows_ν, ω_ν, depth):
2   if ν is a leaf then
3     return
4   VISIT(left child, its rows, its weights, depth+1); VISIT(right child, ...) ; // children first
5   WITH ← CHECK_PATH(tree as it is);
6   MEASURE(REPLAY(SCORES(current leaves), F, 1/n)) ; // holdings for the leaf fit
7   ĉ_ν ← FIT_LEAF(rows_ν, ω_ν, ν.ĉ); make ν a leaf with ĉ_ν, detaching the subtree;
8   WITHOUT ← CHECK_PATH(tree with ν as a leaf);
9   g_t ← WITHOUT.regret_t - WITH.regret_t for t ∈ C; ḡ ← mean(g); se ← std(g)/√|C|;
10  keep ← ḡ > ε_R and ḡ > (k_p se if depth ≥ D_p else 0) and WITH.S > WITHOUT.S + ε_S;
11  if keep then
12    restore the split and its subtree
13  else
14    ν stays a leaf with ĉ_ν
15 VISIT(root, F, 1, 0);
16 function CHECK_PATH(tree):
17   scores ← ROUTE(all rows F ∪ C);
18   u_C ← final holdings of REPLAY(scores_F, F, 1/n); return REPLAY(scores_C, C, u_C) ;
    // fees included; regrets and S on C only

```

---

With  $k_p = 2$ ,  $D_p = 1$ : the root split is judged by the plain rule ( $\bar{g} > \varepsilon_R$  and the Sharpe test); every deeper split must beat two standard errors. A split no checking row reaches has  $\bar{g} = 0$  and is removed.

## 9 Where each quantity lives, and the two facts to remember

|                 |                                                               |                                                                                         |
|-----------------|---------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| per date $t$    | $U[t], B[t], w_t, \text{net}_t, \text{regret}_t$              | written by REPLAY/MEASURE; read by REG; never stored in a node                          |
| per leaf $\ell$ | $\hat{c}_\ell, n_\ell, L_\ell, \text{rows}_\ell, \omega_\ell$ | rows and $\omega$ are training state; the frozen tree keeps $\hat{c}_\ell$ and $n_\ell$ |
| per split node  | $j, \theta$ (soft: $\theta_k, \pi_k$ , spread), children      | no score in use, no holdings                                                            |
| per round       | $S$ , the ranking of candidates                               | recomputed from scratch each round                                                      |

---

- **Fees enter only through REPLAY:**  $\text{fee}_t = f^\top |w_t - u|$  on the path. The regret sees them through  $U[t]$  (frozen for a round) and through  $|w - U[t]|$  inside the utility.

- **Holdings do not move while a candidate split is judged:** the regret gate compares one score against two scores on the same rows from the same  $U$ . The candidate's true path appears only in the Sharpe gate's replay, and becomes the new  $U$  through **MEASURE** when the split is accepted.

## Code map

| block               | function in code/spo_tree.py                                                 |
|---------------------|------------------------------------------------------------------------------|
| MAIN                | SPOPortfolioTree.fit                                                         |
| REPLAY, fees, drift | replay_path, _trade; SPOPortfolioTree._replay, _measure_from                 |
| REG, FIT_LEAF       | _regret_rows, _score_prediction, _fit_leaf                                   |
| GROW, BEST_SPLIT    | _grow, _best_split, _scores, _leaves                                         |
| SHORTLIST, SCREEN   | _shortlist, _screen, _thresholds, _standard_error, _membership_of            |
| REFIT_ALL           | _refit_leaves                                                                |
| PRUNE, CHECK_PATH   | _prune (inner visit, checking_path)                                          |
| optimizer           | PortfolioOptimizer.solve ( $5^n$ enumeration),<br>PortfolioOptimizer.utility |